

Standard Library, struct e C++11

Giada Franz

Volterra, 20 ottobre 2016



- ① C++
- ② C++11
- ③ Containers
- ④ Algoritmi
- ⑤ struct

Variabili e tipi

1 byte = 8 bits

Variabili e tipi

1 byte = 8 bits

char 1 byte 256 caratteri

Variabili e tipi

1 byte = 8 bits

<code>char</code>	1 byte	256 caratteri
<code>bool</code>	1 byte	true o false

Variabili e tipi

1 byte = 8 bits

<code>char</code>	1 byte	256 caratteri
<code>bool</code>	1 byte	<code>true</code> o <code>false</code>
<code>int</code>	4 bytes	circa fra -2 miliardi e 2 miliardi

Variabili e tipi

1 byte = 8 bits

<code>char</code>	1 byte	256 caratteri
<code>bool</code>	1 byte	true o false
<code>int</code>	4 bytes	circa fra -2 miliardi e 2 miliardi
<code>long long int</code>	8 bytes	circa fra $-9 \cdot 10^{18}$ e $9 \cdot 10^{18}$

Variabili e tipi

1 byte = 8 bits

char	1 byte	256 caratteri
bool	1 byte	true o false
int	4 bytes	circa fra -2 miliardi e 2 miliardi
long long int	8 bytes	circa fra $-9 \cdot 10^{18}$ e $9 \cdot 10^{18}$
float	4 bytes	circa nel range $\pm 3,4 \cdot 10^{\pm 38}$ con 7 cifre

Variabili e tipi

1 byte = 8 bits

char	1 byte	256 caratteri
bool	1 byte	true o false
int	4 bytes	circa fra -2 miliardi e 2 miliardi
long long int	8 bytes	circa fra $-9 \cdot 10^{18}$ e $9 \cdot 10^{18}$
float	4 bytes	circa nel range $\pm 3,4 \cdot 10^{\pm 38}$ con 7 cifre
double	8 bytes	circa nel range $\pm 1,7 \cdot 10^{\pm 308}$ con 15 cifre

Tips

- Nelle funzioni evitare di definire localmente array.

Tips

- Nelle funzioni evitare di definire localmente array.
- Possiamo rinominare tipi o funzioni con nomi ingombranti, per esempio:

```
1 using namespace std;
2 using ll = long long int;
3
4 int main() {
5     long long int numero_grande;
6     ll numero_grande_e_comodo;
7 }
```

Tips

- Nelle funzioni evitare di definire localmente array.
- Possiamo rinominare tipi o funzioni con nomi ingombranti, per esempio:

```
1 using namespace std;
2 using ll = long long int;
3
4 int main() {
5     long long int numero_grande;
6     ll numero_grande_e_comodo;
7 }
```

- `cin` e `cout` sono lenti, a meno di usare queste precauzioni:

```
1 cin.tie(0), cin.sync_with_stdio(0);
2 cout.tie(0), cout.sync_with_stdio(0);
```

- È possibile includere direttamente tutte le librerie standard del C++ tramite il comando:

```
#include <bits/stdc++.h>
```

- 1 C++
- 2 C++11
- 3 Containers
- 4 Algoritmi
- 5 struct

auto

È possibile sostituire il tipo di una variabile con `auto`, se il tipo è già chiaro dal contesto.

auto

È possibile sostituire il tipo di una variabile con `auto`, se il tipo è già chiaro dal contesto.

Per esempio

```
set<int>::iterator it = myset.begin();
```

può diventare

```
auto it = myset.begin();
```

Cicli

È possibile ciclare facilmente fra tutti gli elementi di un container. Ad esempio possiamo scrivere

```
1 vector<int> myvector = {2, 7, 3, 4};  
2 for (int v : myvector) cout << v << " ";  
3 cout << endl;
```

Output

```
2 7 3 4
```

Osservazioni

- Quindi in particolare è possibile scrivere

```
set<pair<int, int>> myset;  
for (auto el : myset) {  
    cout << el.first << " " << el.second << endl;  
    ...  
}
```

invece di

```
set<pair<int, int>> myset;  
set<pair<int, int>>::iterator it;  
for (it = myset.begin(); it != myset.end(); it++) {  
    cout << it->first << " " << it->second << endl;  
    ...  
}
```

Attenzione: `el` è una copia del corrispondente elemento di `myset`.

Osservazioni

- Quindi in particolare è possibile scrivere

```
set<pair<int, int>> myset;
for (auto el : myset) {
    cout << el.first << " " << el.second << endl;
    ...
}
```

invece di

```
set<pair<int, int>> myset;
set<pair<int, int>>::iterator it;
for (it = myset.begin(); it != myset.end(); it++) {
    cout << it->first << " " << it->second << endl;
    ...
}
```

Attenzione: `el` è una copia del corrispettivo elemento di `myset`.

- Se si vuole salvare globalmente una variabile locale, è possibile farlo con

```
int N;
void function(int N,...) {
    ::N = N;
    ...
}
```

- ① C++
- ② C++11
- ③ Containers
- ④ Algoritmi
- ⑤ struct

string

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int main(){
6     // Come posso definire una stringa
7     string str1 = "pompelmo";
8     string str2("Girandola17");
9     string str3;
10
11     // empty() ritorna true se la stringa e' vuota
12     if (str3.empty()) {
13         str3 = str1 + str2; // Il + concatena due stringhe
14         // Posso stampare comodamente le stringhe con iostream
15         cout << str3 << endl;
16     }
17
18     // size() ritorna la lunghezza della stringa
19     cout << str1 << ": " << str1.size() << endl;
20 }
```

Output

```
pompelmoGirandola17
pompelmo: 8
```

vector

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int main() {
6     vector<int> array_dinamico; // Semplice vector di interi
7     vector<pair<int, int>> amici[10]; // Array di vector di pair
8
9     if (array_dinamico.empty()) {
10         cout << "Il vector array_dinamico e' ancora vuoto" << endl;
11     }
12
13     amici[0].push_back({2,-1});
14     amici[1].emplace_back(2,10);
15     amici[0].push_back(make_pair(1,2));
16
17     for (int i = 0; i < int(amici[0].size()); i++) {
18         cout << amici[0][i].first << " " << amici[0][i].second << endl;
19     }
20 }

```

Output

```

Il vector array_dinamico e' ancora vuoto
2 -1
1 2

```

stack

```
1 #include <iostream>
2 #include <stack>
3 using namespace std;
4
5 int main() {
6     stack<int> mystack;
7
8     mystack.push(5);
9     mystack.push(3);
10    while (!mystack.empty()) {
11        cout << mystack.top() << endl;
12        mystack.pop();
13    }
14 }
```

Output

```
3
5
```

queue

```
1 #include <iostream>
2 #include <queue>
3 using namespace std;
4
5 int main() {
6     queue<int> myqueue;
7
8     myqueue.push(5);
9     myqueue.push(3);
10    while (!myqueue.empty()) {
11        cout << myqueue.front() << endl;
12        myqueue.pop();
13    }
14 }
```

Output

```
5
3
```

deque

```
1 #include <iostream>
2 #include <deque>
3 using namespace std;
4
5 int main() {
6     deque<int> mydeque;
7
8     mydeque.push_back(5);
9     mydeque.push_back(3); // mydeque = {5, 3}
10
11     cout << mydeque.front() << " " << mydeque.back() << endl;
12
13     mydeque.pop_front(); // mydeque = {3}
14
15     mydeque.push_front(10); // mydeque = {10, 3}
16
17     while (!mydeque.empty()) {
18         cout << mydeque.front() << " ";
19         mydeque.pop_front();
20     }
21     cout << endl;
22 }
```

Output

```
5 3
10 3
```

priority_queue

```
1 #include <iostream>
2 #include <queue>
3 using namespace std;
4
5 int main() {
6     priority_queue<int> mypq;
7
8     mypq.push(5);
9     mypq.push(3);
10    mypq.push(10);
11    mypq.push(4);
12
13    while (!mypq.empty()) {
14        cout << mypq.top() << " ";
15        mypq.pop();
16    }
17    cout << endl;
18 }
```

Output

```
10 5 4 3
```

set

```
1 #include <iostream>
2 #include <set>
3 using namespace std;
4
5 int main() {
6     set<int> myset;
7
8     myset.insert(30);
9     myset.insert(10); // myset = {10, 30}
10    myset.erase(10); // myset = {30}
11
12    int new_elements[] = {30, 43, 20, 23};
13    myset.insert(new_elements, new_elements+4); // myset = {20, 23, 30, 43}
14
15    for (int x : myset) {
16        cout << x << " ";
17    }
18    cout << endl;
19
20    auto itlow = myset.lower_bound(22);
21    cout << *itlow << endl;
22 }
```

Output

```
20 23 30 43
23
```

map

```
1 #include <iostream>
2 #include <map>
3 using namespace std;
4
5 int main() {
6     map<string, int> mymap;
7
8     mymap["mele"] = 7;
9     mymap["ciliegie"] = 5;
10    mymap.insert({"banane", 2});
11
12    mymap["mele"]--;
13
14    auto it_ciliegie = mymap.find("ciliegie");
15    if (it_ciliegie != mymap.end()) mymap.erase(it_ciliegie);
16
17    for (const auto& x : mymap) {
18        cout << x.first << " => " << x.second << endl;
19    }
20 }
```

Output

```
banane => 2
mele => 6
```

unordered_set

Posso solo inserire, rimuovere, cercare. Non ho più niente sull'ordinamento.

```
1 #include <iostream>
2 #include <unordered_set>
3 using namespace std;
4
5 int main() {
6     unordered_set<int> myuset;
7
8     myuset.insert(30);
9     myuset.insert(10); // myuset = {30, 10}
10
11     int new_elements[] = {30, 43, 20, 23};
12     myuset.insert(new_elements, new_elements+4); // myuset = {23, 30, 43, 10}
13
14     myuset.erase(20); // myuset = {30, 10, 43, 23}
15
16     for (int x : myuset) {
17         cout << x << " ";
18     }
19     cout << endl;
20 }
```

Output

```
23 30 43 10
```

Osservazioni

- `unordered_map` è il corrispettivo di `unordered_set` per le map.

Osservazioni

- `unordered_map` è il corrispettivo di `unordered_set` per le map.
- Posso utilizzare come costruttore di un set un vector o viceversa

```
1 #include <vector>
2 #include <set>
3 using namespace std;
4
5 int main() {
6     vector<int> myvector = {5, 3, 6};
7
8     set<int> myset(myvector.begin(), myvector.end());
9
10    vector<int> secondvector(myset.begin(), myset.end());
11 }
```

e lo stesso vale per gli altri containers.

- ① C++
- ② C++11
- ③ Containers
- ④ Algoritmi**
- ⑤ struct

sort

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 int main() {
7     vector<int> myvector = {5, 3, 6};
8
9     sort(myvector.begin(), myvector.end());
10    for (int i = 0; i < int(myvector.size()); i++) {
11        cout << myvector[i] << " ";
12    }
13    cout << endl;
14 }
```

Output

```
3 5 6
```

sort con compare

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 bool compare(int i, int j) {
7     return i>j;
8 }
9
10 int main() {
11     vector<int> myvector = {5, 3, 6};
12
13     sort(myvector.begin(), myvector.end(), compare);
14     for (int i = 0; i < int(myvector.size()); i++) {
15         cout << myvector[i] << " ";
16     }
17     cout << endl;
18 }
```

Output

```
6 5 3
```

lower_bound

lower_bound non funziona solo su set o su map, ma su tutte le strutture ordinate.

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int main() {
6     vector<int> myvector;
7     for (int k = 0; k < 5; k++) myvector.push_back(10*k);
8     // myvector = {0, 10, 20, 30, 40}
9
10    auto it = lower_bound(myvector.begin(), myvector.end(), 20);
11
12    cout << *it << endl;
13 }
```

Output

```
20
```

lower_bound

`lower_bound` non funziona solo su `set` o su `map`, ma su tutte le strutture ordinate.

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int main() {
6     vector<int> myvector;
7     for (int k = 0; k < 5; k++) myvector.push_back(10*k);
8     // myvector = {0, 10, 20, 30, 40}
9
10    auto it = lower_bound(myvector.begin(), myvector.end(), 20);
11
12    cout << *it << endl;
13 }
```

Output

```
20
```

Anche in questo caso è possibile passare una funzione `compare`, ma il container deve essere già ordinato rispetto a tale funzione.

next_permutation

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 int main() {
7     vector<int> perm = {4, 3, 5};
8
9     while (next_permutation(perm.begin(), perm.end())) {
10         cout << perm[0] << " " << perm[1] << " " << perm[2] << endl;
11     }
12 }
```

Output

```
4 5 3
5 3 4
5 4 3
```

- ① C++
- ② C++11
- ③ Containers
- ④ Algoritmi
- ⑤ **struct**

struct semplice

```
1 #include <iostream>
2 using namespace std;
3
4 struct Bambino {
5     int peso;
6     int altezza;
7 };
8
9 int main() {
10     Bambino a, b;
11
12     a.peso = 30;
13     a.altezza = 50;
14
15     b.peso = 7;
16     b.altezza = 5;
17
18     cout << a.altezza << endl;
19 }
```

Output

```
50
```

Costruttori

```
1 #include <iostream>
2 using namespace std;
3
4 struct Bambino {
5     int peso, altezza;
6
7     Bambino () {
8         peso = 0;
9         altezza = 0;
10    }
11    Bambino (int p, int a) {
12        peso = p;
13        altezza = a;
14    }
15 };
16
17 int main() {
18     Bambino a = Bambino();
19     Bambino b = Bambino(5, 3);
20     cout << a.peso << " " << a.altezza << endl;
21     cout << b.peso << " " << b.altezza << endl;
22 }
```

Output

```
0 0
5 3
```

Puntatori

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 struct Vertice {
6     int id;
7     vector<Vertice*> figli;
8 };
9
10 int main() {
11     vector<Vertice*> albero;
12     for (int i = 0; i < 5; i++) {
13         albero.push_back(new Vertice());
14     }
15 }
```

In questo caso l'uso dei puntatori è necessario, ma in generale cercare di evitarli.

Metodi

```
1 #include <iostream>
2 using namespace std;
3
4 struct Bambino {
5     int peso, altezza;
6
7     Bambino (int p, int a) {
8         peso = p;
9         altezza = a;
10    }
11
12    void somma(Bambino other) {
13        peso += other.peso;
14        altezza += other.altezza;
15    }
16 };
17
18 int main() {
19     Bambino a = Bambino(30, 50);
20     Bambino b = Bambino(7, 5);
21     a.somma(b);
22     cout << a.peso << " " << a.altezza << endl;
23 }
```

Output

```
37 55
```

Operatori

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 #include <set>
5 using namespace std;
6
7 struct Bambino {
8     int peso, altezza;
9
10    Bambino (int p, int a) {
11        peso = p;
12        altezza = a;
13    }
14    bool operator<(const Bambino& other) const {
15        if (peso != other.peso) return (peso < other.peso);
16        return (altezza < other.altezza);
17    }
18 };
19
20 int main() {
21     vector<Bambino> bimbi = {Bambino(30, 50), Bambino(7, 5),
22                             Bambino(23, 8)};
23     sort(bimbi.begin(), bimbi.end()); // {(7, 5), (23, 8), (30, 50)};
24     set<Bambino> bimbi_set;
25     bimbi_set.insert(Bambino(30, 50));
26 }
```

Esempio: geometria

Esercizio

Scrivere una struct Punto che rappresenta i punti del piano e una struct Retta che rappresenta le rette, che implementano le seguenti funzionalità:

- *ordinamento “rotante”;*
- *sapere se un punto appartiene ad una retta;*
- *trovare la retta per due punti;*
- *trovare l'intersezione fra due rette.*